APPENDIX

This appendix gives the model behind Sections III to VI and full proofs of the results stated there. Results of the main text keep their numbers, and results that appear only here are numbered after them.

A. Model

a) Ledger and state: A *ledger* $L = T_1, T_2, \ldots$ is the total order fixed by block number and position in block. A *state* maps every key k to a pair $(val(k), ver(k))$, or to $\perp$ if k has never been written. Every write to a key increments its version, and a new key starts at version 0, so along a ledger the version of a key identifies its value. Each transaction is executed, and its content evaluated, with the block context (timestamp, number) of its endorsement. Executing T on a state means running its contract call in the EVM on that state with this context.

b) Signed content: The signed content of a transaction T consists of a read set $R(T)$, which pairs every key T reads with the version it read, plain writes $W(T)$ of pairs (k, x), guarded deltas $\Delta(T)$ of tuples (k, δ, ℓ, u), and at most one program $P(T) = (K, N, G, O)$ as defined in §V-B. The keys written by the three kinds are disjoint. Content certifies an outcome only if its endorsements satisfy the endorsement policy. The keys of $\Delta(T)$ and K are *covered*, written $\mathrm{cov}(T)$, and EPR does not version-check reads of covered keys. Every storage slot that a program reads is an input in K, so the uncovered reads of a program transaction are reads of account fields such as balances and sequence numbers.

c) Delta guards: The guard of a delta (k, δ, ℓ, u) on a value x of k is

$$0 \le x \le 2^{63} - 1 \quad \text{and} \quad \ell \le x + \delta \le u.$$

The tracer of §V-A keeps an interval $[lo_k, hi_k]$ of base values that starts as the int64 domain $[0, 2^{63} - 1]$. It sets $\ell = lo_k + \delta$ and $u = hi_k + \delta$, except that $u = 2^{63} - 1$ when $hi_k = 2^{63} - 1$. The no-wrap constraint of the written value gives $\ell \ge 0$ and $hi_k + \delta \le 2^{63} - 1$, so in both cases the guard holds on x exactly when $x \in [lo_k, hi_k]$. A delta is a *credit* if $\delta \ge 0$, $\ell \le \delta$ and $u = 2^{63} - 1$, and a *debit* if $u = 2^{63} - 1$ and it is not a credit. Deltas with $u < 2^{63} - 1$ are ordered like programs. Credits and the debits that escrow admits on the fast path form the *commutative class*. Two accesses to the same key by different transactions *conflict* unless both are in the commutative class.

d) Guard and evaluation: In the terms of §III, the guard $g_T(s)$ of the signed content of T holds iff $ver_s(k) = v$ for every $(k, v) \in R(T)$ with $k \notin \mathrm{cov}(T)$, the guard of every delta of T holds on $val_s(k)$, and every guard in G holds when N is evaluated on s. The evaluation $e_T(s)$ writes x to k for every $(k, x) \in W(T)$, $val_s(k) + \delta$ to k for every $(k, \delta, \ell, u) \in \Delta(T)$, and $O(k)$, evaluated on s, to every key k of O.

Definition 2 (Serial semantics)**.** *Starting from the genesis state, $S(L)$ processes L in order. With current state s, a transaction T is* valid *iff its endorsements satisfy the policy and $g_T(s)$ holds. A valid T applies $e_T(s)$ at once and increments the version of every key it writes. An invalid T leaves s unchanged.*

$S(L)$ is defined over ledger state. The logs, return data and gas use reported for a repaired program are those of its endorsement. Theorem 2 concerns EPR without the query rule of §VI, that is, with every uncovered read version-checked, and Proposition 2 adds the rule.

e) Assumptions: We rely on the following properties of Fabric-X and of our implementation.

- A1 *Dependency graph.* If T_i precedes T_j in L and they conflict on some key, T_j is dispatched for validation only after T_i is *final*, that is, committed or invalidated.
- A2 *Batch order.* A VC evaluates the transactions of a batch in ledger order (Algorithm 2).
- A3 *Atomic commit.* The writes, increments and statuses of a batch commit in one database transaction. An increment of a key by the sum of n deltas adds n to its version in the same statement, or inserts the key at version $n - 1$ if it does not exist, and the row lock serializes concurrent increments of a key.
- A4 *Range.* No credit or debit overflows int64: on every value x of its key on which a credit or debit may be evaluated or applied, in $S(L)$ or in EPR, $0 \le x \le 2^{63} - 1$ and $x + \delta \le 2^{63} - 1$.
- A5 *Same evaluator.* Endorsers, committers and endorser replicas evaluate signed content with one deterministic evaluator whose operators implement the `uint256` semantics of the EVM [64].
- A6 *Value domain.* Every value of a key that escrow classifies is at least a declared bound D, with $D = 0$ for EVM storage words, which are unsigned.
- A7 *Gas headroom.* At every opcode of T other than a call or a create, the gas left after the opcode in the endorsed run is at least 20,000 gas per SSTORE executed so far and, for a program, 1,600 gas per EXP executed so far. No frame of the endorsed run runs out of gas after the first SSTORE or EXP with a shadowed exponent, and in a program the remaining gas serves only as the gas operand of a call, raw or offset by constants below 2^{64} that keep it non-negative in the endorsed run. Endorsers check these conditions and keep plain writes otherwise.
- A8 *Consistent snapshot.* Endorsers simulate T on the committed state after the last transaction of one block h, the *endorsement height* of T, which precedes T in L. Endorser replicas reach this state by applying the committed blocks in order with the evaluator of A5.
- A9 *Sequence-number discipline.* Only transactions of a sender write its sequence number, and each either reads it under a version check or writes it without reading it (§VIII-B).

A4 is the condition of Theorem 2 that no credit or debit overflows int64 in $S(L)$ or in EPR. It holds for a key whenever its initial value plus every increase that transactions of L may make to it stays below 2^{63}, since nothing else raises it. For a token whose transactions only move balances between

accounts, it also holds whenever the sum of all balance words is below 2^{63}. By induction over commits, every database state consists of whole transactions (A3), which preserve this sum, and holds no negative balance, since every debit is applied only on values on which its guard holds (Theorem 1 for fast-path debits, exact evaluation for the others). Hence no balance exceeds the sum.

The costs that can depend on the storage values T reads are those of SSTORE, which depend on the original, current and new values of the slot [63] and differ by at most 19,900 gas (20,000 against 100), and of EXP, which grows by 50 gas per byte of its exponent and differs by at most 1,600 gas. The proofs of Lemmas 1 and 2 show that every other dynamic cost is equal in both runs, and the gas operand of a call is unshadowed, guarded, or the remaining gas offset by constants as A7 requires.

Lemma 3 (Gas headroom)**.** *Under A7, on every state on which g_T holds, a call frame of T runs out of gas exactly where it does in the endorsed run.*

Proof. Until the first SSTORE or EXP with a shadowed exponent, every cost, and hence the remaining gas, is equal in both runs, so the claim holds there. From then on, the endorsed run never runs out of gas (A7), and we show that the run on such a state s' does not either. Let E_k be the bound of A7 at opcode k of the endorsed run, which never decreases. By induction over the opcodes, the gas available before opcode k on s' is at least the endorsed one minus E_{k-1}. Within a frame this follows since opcode k costs at most $E_k - E_{k-1}$ more on s'. A callee receives $\min(c, a - \lfloor a/64 \rfloor)$, where a is the caller's gas after the base cost of the call and c the gas operand; on s' these are c' and $a' \geq a - E_{k-1}$. Either c is an unshadowed or guarded word, equal in both runs, or, in a program, $c \equiv r + d \pmod{2^{256}}$, where r is the remaining gas read at an earlier opcode, d the sum of the constants added to or subtracted from it, each below 2^{64}, and $r + d \geq 0$ (A7). In the second case $r' \geq r - E_{k-1}$, so either $c' = r' + d \geq c - E_{k-1}$, or $r' + d < 0$, and then $c' > 2^{255}$, since programs have at most 4,096 nodes, and the callee receives all of $a' - \lfloor a'/64 \rfloor$. As $a' - \lfloor a'/64 \rfloor \geq a - \lfloor a/64 \rfloor - E_{k-1}$ and the minimum is monotone, the allowance shrinks by at most E_{k-1}. A returning callee hands its remainder back, so the bound carries over to the caller. By A7, the endorsed gas before opcode k is at least its endorsed cost plus E_k, so on s' it covers the cost of opcode k. The base cost of a call or create is covered by the caller's next opcode, whose gas never exceeds the gas left after that base cost. The SSTORE condition of EIP-2200 that more than 2,300 gas remain holds since $E_k \geq 20{,}000$ at an SSTORE. Refunds apply only after execution. □

B. Extraction

Lemma 1 (Delta soundness)**.** *Let T be endorsed on state s, let Σ be the set of slots for which the tracer emits a delta $(\sigma, \delta, \ell, u)$, and assume A7. For every state s' that agrees with s on every key T reads outside Σ and on which the guard of every delta of T holds, executing T on s' follows the endorsed opcode sequence, writes $x'_\sigma + \delta$ to every $\sigma \in \Sigma$, and writes the endorsed value to every other key.*

Proof. Write x_σ and x'_σ for the values of σ in s and s'. Both lie in $[lo_\sigma, hi_\sigma] \subseteq [0, 2^{63} - 1]$, the first because the tracer extracts a delta only if the endorsed run satisfies its constraints, the second by the guard. A label *involves* σ if it is $O(\sigma)$, $D(\sigma)$, $G(\sigma)$, or an X label derived from σ. We show by induction over the executed opcodes that both runs execute the same opcode and that every word w (stack, memory, call data, return data) is in one of four classes. (a) Its label involves no slot of Σ, and w has equal values in both runs. (b) It is labeled $O(\sigma)$ or $D(\sigma)$ with $\sigma \in \Sigma$ and an offset c that is equal in both runs, and it has the values $x_\sigma + c$ and $x'_\sigma + c$, both in $[0, 2^{63} - 1]$. (c) It is labeled $G(\sigma)$ with $\sigma \in \Sigma$, and has equal values. (d) It is labeled X and involves a slot of Σ, and no leaking operation uses it. The leaking operations of line 9 of Algorithm 1 are the uses of a labeled word in control flow other than as the G condition of a conditional jump, in memory, logs, calls, hashing, return data, transient storage or an EXP exponent, as a storage key, and as a value stored to a slot other than its own. Each marks the slots its word involves as leaked, and slots of Σ are not leaked. Words of classes (b) to (d) therefore stay on the stack, and memory, call data and return data hold class-(a) words only. Initially every word comes from the transaction or from the block context of its endorsement and is in class (a).

Dependencies. When a word whose label involves another slot σ' enters an offset or a comparison of σ, the tracer makes σ' a dependency of σ. It extracts σ only if T does not write σ', whose read then stays version-checked, and it never extracts a slot on which another slot depends. Hence the dependencies of slots of Σ lie outside Σ and agree in both runs, and every offset and every operand compared with a class-(b) word below is a class-(a) word.

Loads. The key of an SLOAD is a class-(a) word, so both runs read the same slot. A slot of Σ is not accessed after T writes it, so it is read before, and the result is $O(\sigma)$ with $c = 0$, in class (b). Any other slot either has not been written by T and agrees in both runs by assumption, or holds a class-(a) word that T wrote. Reads of account fields such as balances and code are reads of keys outside Σ and agree as well.

Addition and subtraction of a class-(b) word $x_\sigma + c$ and a class-(a) word b give $x_\sigma + c \pm b$ in the integers if no wrap occurs. The tracer records the no-wrap constraint $0 \leq x_\sigma + c \pm b \leq 2^{63} - 1$, which every base value in $[lo_\sigma, hi_\sigma]$ satisfies. It holds for x_σ and x'_σ alike, modular arithmetic coincides with integer arithmetic in both runs, and the result is in class (b) with offset $c \pm b$.

Masking and shifts. AND with a constant whose low 63 bits are set is the identity on $[0, 2^{63})$, and SHR by at least 63 bits maps this range to 0. Class-(b) words lie in this range in both runs, so the first result stays in class (b), and the second is 0 in both runs, in class (a). A balance word of USDC v2.2 whose blacklist bit, bit 255, is set lies outside $[0, 2^{63} - 1]$, so the guard fails on it, just as the transfer reverts.

Comparisons. Let one operand be a class-(b) word $x_\sigma + c$, the other a class-(a) word b, and let r be the endorsed outcome. The word $x_\sigma + c$ increases with x_σ and lies in $[0, 2^{63} - 1]$, where its signed and unsigned values coincide. The base values on which the comparison yields r therefore form a half-line for an order comparison, a point for a true equality, and the complement of a point for a false equality. The tracer intersects the interval with the half-line or the point. A false equality whose excluded value is an end of the interval cuts that end off, and one whose excluded value lies strictly inside prevents the delta. Hence the comparison yields r on x'_σ as well, and the result is in class (c). ISZERO of a class-(b) word is the comparison with 0, and ISZERO of a class-(c) word is again in class (c). If both operands are class-(b) words of the same slot, with offsets c_1 and c_2, both lie in range and no wrap occurs, so the outcome depends on c_1 and c_2 only and is equal in both runs. Every other operation with an operand in classes (b) to (d) yields an X word, in class (d).

Control flow. The condition of a conditional jump is in class (a) or (c), and jump targets are in class (a), so both runs take the same branch.

Stores. The key of an SSTORE is a class-(a) word. A write to $\sigma \in \Sigma$ stores a class-(b) word of σ with offset δ, that is, $x'_\sigma + \delta$ on s'. Every other write stores a class-(a) word, equal in both runs.

Gas. Labeled words reach no offset, size, call argument or exponent, so every dynamic cost other than that of an SSTORE to a slot of Σ is equal in both runs. Such a slot is read before it is written and written once, so its original and current values are equal at the write, and its new value differs from them in both runs if $\delta \neq 0$ and in neither if $\delta = 0$. By EIP-2200 [63] the write costs 20,000 or 2,900 gas, depending on whether the original value, which may differ between the runs, is zero, or 100 gas in both, and cold-access surcharges are equal. The remaining gas therefore differs by at most 17,100 gas per extracted slot written so far. It is observable only through GAS, after which the tracer abandons the extraction of every slot written so far, and through running out of gas, which Lemma 3 excludes. Refunds apply only after execution.

Reverts occur at the same instructions in both runs and discard the same writes. The invariant holds after every opcode, which proves the claim. □

Corollary 1 (Weakest delta guard)**.** *For an extracted slot σ, the set $[lo_\sigma, hi_\sigma]$ of base values on which its guard holds is exactly the set of $x'_\sigma \in [0, 2^{63} - 1]$ on which every comparison on σ, that is, every comparison whose result the tracer labels $G(\sigma)$, keeps its endorsed outcome and every class-(b) word $x'_\sigma + c$ lies in $[0, 2^{63} - 1]$.*

Proof. The interval is the int64 domain intersected with the constraints the tracer recorded. A comparison constraint is exactly the set of base values on which that comparison yields its endorsed outcome. For a false equality, the tracer extracts a delta only if this set, intersected with the interval, is again an interval, and then records exactly that intersection. A no-wrap constraint is exactly the set on which $x'_\sigma + c$ lies in $[0, 2^{63} - 1]$. Comparisons of two offsets of the same slot add no constraint, since their outcome is the same on every base value that satisfies the no-wrap constraints. □

Lemma 2 (Program soundness)**.** *Let T be endorsed on state s with program $P = (K, N, G, O)$, and assume A5 and A7. For every state s' that agrees with s on every key T reads outside K and on which every guard in G holds, with N evaluated on s', executing T on s' follows the endorsed opcode sequence, writes $O(k)$ evaluated on s' to every key k of O, and writes the endorsed value to every other key.*

Proof. We show by induction over the executed opcodes that both runs execute the same opcode, that every shadowed location holds on s' the evaluation on s' of its node, and that every other location holds the same value in both runs. The locations are stack slots, 32-byte memory words, call-data and return-data words in every frame, and the storage and transient-storage slots that T has written. Initially nothing is shadowed, and both runs execute the same transaction with the same block context.

Loads. SLOAD with a shadowed key carries an equality guard, so both runs read the same slot. If T has not written it, the result is the input node of the slot, whose evaluation on s' is its value there. If T has written it, the tracer forwards the node of that write, which holds the stored value by the induction hypothesis. TLOAD is handled in the same way for transient storage. Reads of account fields such as balances and code are reads of keys outside K, which agree by assumption.

Operators with a shadowed operand create a node whose evaluation equals the EVM result, by A5.

Control flow. A shadowed branch condition carries a truthiness guard and a shadowed jump target an equality guard, so both runs take the same path.

Memory, call data and return data. Shadowed offsets and sizes carry equality guards, so both runs access the same locations. Whole-word copies preserve the invariant. A call input that ends inside a shadowed word passes the word masked to its leading bytes, as the EVM pads call data with zeros. A load that overlaps a shadowed word composes a node from the shifted shadowed word and the concrete bytes. A store that partially overwrites a shadowed word poisons its remaining bytes, and the tracer abandons extraction if poisoned bytes are read.

External effects. Shadowed storage keys, call targets, call values, account addresses and inputs of hashes and precompiles carry equality guards. These operations therefore see equal inputs in both runs and return equal, unshadowed results.

Reverts. A frame reverts in both runs alike, since the path is the same and Lemma 3 excludes running out of gas. Its writes to storage and to transient storage are then rolled back from the journal of the tracer, as in the EVM.

Gas. Every dynamic cost other than those of SSTORE and EXP depends on offsets, sizes, call targets, call values or account fields, which are equal in both runs. The remaining gas is therefore equal in both runs until the first SSTORE or EXP with a shadowed exponent, since a call gas operand computed

from shadowed values is guarded. From then on, the output of GAS is shadowed by a marker that may only flow, raw or offset by constants below 2^{64} that keep it non-negative, into the gas operand of a CALL, CALLCODE, DELEGATECALL or STATICCALL, and the tracer abandons extraction if it reaches any other use. A gas operand computed from shadowed values is guarded, and one computed from the marker changes the callee's allowance by at most the extra gas paid before the call, which Lemma 3 accounts for.

Hence the invariant holds after every opcode, and every SSTORE writes the evaluation of its node. The final storage writes are O evaluated on s', and the unshadowed writes, such as the sequence number, are the endorsed ones. □

C. Serial equivalence and determinism

Theorem 2 (Serial equivalence)**.** *Under A1 to A6, where A4 requires that no credit or debit overflows int64, the committed state (values and versions) and the validity of every transaction of L equal those of $S(L)$.*

Proof. We show by induction over the position of T in L that EPR decides T as $S(L)$ does, and then that the committed state is that of $S(L)$. Assume that every transaction before T is decided as in $S(L)$, and let k be a key that T accesses.

Ordered accesses. Let the access of T to k lie outside the commutative class: a read, a plain write, a debit on the ordered path, another delta, or a program input or output. It conflicts with every access to k by another transaction. By A1, every earlier transaction that accesses k is final when T is dispatched, and no later one is dispatched before T is final. The value and version of k that the VC uses for T therefore result from exactly the valid accesses to k before T, which by the induction hypothesis are those of $S(L)$. Their effects compose as in $S(L)$: accesses outside the commutative class are applied one at a time in ledger order, and the increments between two of them commute, by the argument on the state below, which uses only the decisions on these earlier transactions. A3 makes every batch visible at once, and A2 orders the evaluation within a batch. Hence the VC evaluates every version check, guard and output of T with the values and versions that $S(L)$ gives the keys of T before T, that is, on the pre-state of T.

Commutative accesses. The guard of a credit holds on every value x with $0 \le x \le 2^{63} - 1$ and $x + \delta \le 2^{63} - 1$, since $\ell \le \delta$. By A4 these conditions hold on every value on which the credit is evaluated or applied, so its guard holds in $S(L)$ and in EPR alike. The guard of a fast-path debit holds in $S(L)$ and on every value of k at which the VC may evaluate or apply it, by Theorem 1, whose proof below uses only the induction hypothesis. In both cases the outcome of the guard is that of $S(L)$, whatever value the VC reads.

Validity. By the two cases, every version check and guard of T has its outcome in $S(L)$, and EPR checks the endorsement policy as $S(L)$ does. Hence T is valid iff it is valid in $S(L)$, which completes the induction. An invalid T contributes no write, including its increments, since the VC aggregates increments over the valid transactions only.

State. Fix a key k. In $S(L)$, a valid transaction writes the endorsed value of a plain write, $val(k)+\delta$ for a delta, or $O(k)$ evaluated on its pre-state. EPR computes the writes outside the commutative class on the exact pre-state, by the first case. It applies the increments of the commutative class in some order between the accesses outside the class that surround them in L. Integer addition commutes, A4 keeps every intermediate value in range, and A3 makes each increment atomic, so after every access outside the class, and at the end, k holds its value in $S(L)$. In $S(L)$, every valid write to k advances its version by one. EPR advances a key outside the commutative class by the number of valid writers in the batch (Algorithm 2, lines 8 and 9), and an increment by n deltas advances it by n or inserts the key at version $n - 1$ (A3). The resulting version counts the valid writes to k, independently of batching and of the order of increments, as in $S(L)$.

Optimizations of §VI-A. The endorsed fast path skips the evaluation only when every read of T is current. Every value T reads is then the endorsed one, on which the guards hold, and the evaluation would return the endorsed writes, which the fast path installs, with a delta applied as an increment. The concurrent value fetch and the pipelined repair stage overlap batches only on keys of the commutative class, since A1 releases any other conflicting access only after its predecessors are final, and the commutative case above does not depend on the value read. □

Definition 3 (Query)**.** *A transaction is a* query *if its signed content writes only its sender's sequence number. Under the query rule of §VI,* EPR *skips the version checks of the contract-storage reads of a query, and of no other read.*

Let L° be L with the contract-storage reads of every query removed from its read set. Let L' be obtained from L by moving the contract-storage reads of every query Q with endorsement height h to just after the last transaction of block h, while the other reads of Q, including any read of its sequence number, and its writes keep the position of Q. In $S(L')$, Q is valid iff its endorsements satisfy the policy and each of its reads passes its version check at its position in L', and its writes take effect at the position of Q.

Proposition 2 (Queries at their snapshot)**.** *Under A1 to A6, A8 and A9, the state and validity decisions of* EPR *with the query rule equal those of $S(L')$. In $S(L')$, every query reads contract storage in the state after its endorsement height, a committed prefix of the ledger, and writes the sequence number that executing it at its ledger position writes.*

Proof. Step 1. Under the query rule, the VC treats a query as EPR without the rule treats the same transaction of L°: it checks the same reads and evaluates the same content. The skipped reads only add dependency edges, which A1 allows. By Theorem 2 applied to L°, the state and validity decisions of EPR are those of $S(L^\circ)$.

Step 2. $S(L')$ and $S(L^\circ)$ differ only in the moved reads, which change no state. They make the same decisions if every moved read passes its version check in $S(L')$. We show by

induction over the ledger that they decide every transaction alike. If they decide all transactions before a query Q with endorsement height h alike, their states after block h are equal, since moved reads change no state. By Step 1, A5 and A8, the state of $S(L^\circ)$ after block h is the committed state on which Q was simulated, so every version in the read set of Q is current there, and the moved reads of Q pass. The other reads of Q meet equal states in both, so Q is decided alike. A transaction other than a query also meets equal states in both and is decided alike. Hence $S(L')$ and $S(L^\circ)$, and with Step 1 EPR, have the same state and decisions.

Sequence number. The accesses of Q to its sequence number n did not move, so every other transaction meets them in the same order as in L. By A9, if Q reads n, that read is version-checked at the position of Q, so the sequence number Q writes is computed from the value that $S(L')$ gives n there. If Q writes its client-assigned sequence number without reading it, that value is state-independent (§VIII-B). In both cases Q writes the sequence number that executing it at its ledger position writes, while its contract-storage reads return the state after block h. □

Theorem 3 (Determinism)**.** *Under A1 to A6, committers of different organizations that process the same ledger reach the same state and validity decisions, regardless of batching, scheduling, timing, number of VCs and escrow state, with or without the query rule.*

Proof. The proof of Theorem 2 assumes nothing about batching, scheduling, timing or the number of VCs beyond A1 to A3, and nothing about the escrow state beyond the classification rules of Algorithm 3, formalized below. Fast-path debits, like credits, are applied as increments. By Theorem 1 their guards hold in $S(L)$ and in every interleaving, so their values and versions compose as those of credits in Theorem 2. Every committer therefore reaches the state and validity decisions of $S(L)$, a function of L alone. With the query rule, it reaches those of $S(L^\circ)$ by Step 1 of the proof of Proposition 2, again a function of L alone. □

Proposition 1 (Per-transaction maximality)**.** *Let* Π *be an endorsement-preserving protocol that uses, for every transaction of* L*, the same signed content as* EPR*. If* Π *and* EPR *decide all transactions before* T *alike and* Π *commits* T*, then* EPR *commits* T*. Moreover,* EPR *commits every transaction that EOV commits on the same pre-state.*

We read Definition 1 with states that carry versions as in $S(L)$, where every committed write advances the version of its key by one.

Proof. Equal decisions on the transactions before T and serial equivalence give T the same pre-state s, values and versions, under both protocols. For a query, this pre-state consists of its snapshot for contract storage and of the state at its ledger position for its other keys (Definition 1). Π commits T, so by certified outcomes the outcome of T on s is certified: the endorsements of T satisfy the policy and $g_T(s)$ holds. By Theorem 2 and Proposition 2, EPR decides T as $S(L')$ does, where T is valid iff its endorsements satisfy the policy and $g_T(s)$ holds. Hence EPR commits T.

EOV commits T on s only if every key T read still has the version read at endorsement, and hence the endorsed value. The tracer records only constraints that the endorsed values satisfy, so every version check and every delta and program guard, which held at endorsement, holds on s. Hence $g_T(s)$ holds, and EPR commits T by the first part. □

Proposition 1 is per transaction; §VIII compares whole ledgers through throughput and abort rates.

Corollary 2 (EPR is endorsement-preserving)**.** *Under A1 to A9, the signed content of* EPR *is path-faithful on every state reached along* L*, and* EPR *is endorsement-preserving (Definition 1). For every committed transaction, executing it on its pre-state follows the endorsed path and writes what* EPR *commits.*

Proof. Path-faithful content. Let T be endorsed on state s_e, and let $g_T(s)$ hold on a state s reached along L. Every uncovered read of T has its endorsed version on s, hence its endorsed value. If T has deltas, Lemma 1 applies with Σ the slots of its deltas, since s agrees with s_e on every other key T reads, and T follows the endorsed path and writes $e_T(s)$. If T has a program, Lemma 2 gives the same, with A5. Otherwise every key T reads has its endorsed value, and the EVM, run with the same transaction and block context, repeats the endorsed run.

Endorsement-preserving commit. Serial equivalence is Theorem 2 together with Proposition 2. EPR commits T only when the VC finds g_T to hold, which by Theorem 2 is on the pre-state of T, and it installs e_T evaluated there, so every committed outcome is certified. Determinism is Theorem 3.

Re-execution. By path-faithfulness, the writes that EPR commits for T are those of executing T on its pre-state. By induction over L, the committed values equal those reached by executing the committed transactions in ledger order, each with the block context of its endorsement and with queries reading contract storage at their endorsement height. □

D. Escrow

The coordinator handles the accesses to a key k in ledger order (Algorithm 3). An access is *final* once its transaction is. The coordinator keeps $prefix_k$, the value of k after the longest prefix of its accesses that are final, the list P_k of later deltas and barriers, a flag $unknown_k$, and ℓ_k^*, the largest lower bound among the debits on k classified so far, which never decreases. A *barrier* is an access to k that is neither a read, a credit nor a debit, that is, a plain write, a program output or a delta with $u < 2^{63} - 1$. A debit d on k with amount δ_d and bound ℓ_d is classified when it arrives: $\ell_k^* \leftarrow \max(\ell_k^*, \ell_d)$ and

$$LB(d) = prefix_k + \sum_{i \in F_k} \delta_i + \sum_{i \in U_k} \min(\delta_i, 0),$$

where F_k is the set of final valid deltas in P_k and U_k the set of debits in P_k that are not final. Credits in P_k that are not

final count as 0. The debit d takes the fast path iff $unknown_k$ is not set, no barrier is in P_k and $LB(d) + \delta_d \geq \ell_k^*$, and the ordered path otherwise. When the first access in P_k is final, the coordinator absorbs it into $prefix_k$. A valid delta adds its δ, and an invalid access adds nothing. A valid plain write of an integer sets $prefix_k$ to that integer and clears $unknown_k$, and any other valid barrier sets $unknown_k$.

Lemma 4 (Prefix)**.** *Assume A6, and that every transaction whose access to k has been absorbed is decided as in $S(L)$. Whenever $unknown_k$ is not set and no barrier is in P_k, $prefix_k$ is at most the value of k in $S(L)$ after the absorbed accesses.*

Proof. When the coordinator starts to track k, at genesis, on its first access or after a restart, it sets $prefix_k = D$, which by A6 is at most the value of k at that point of $S(L)$. Absorbing an access changes $prefix_k$ by its effect in $S(L)$: a valid delta adds δ to both sides, an invalid access changes neither, and a valid plain write of an integer sets both sides to that integer. These outcomes are those of $S(L)$ by assumption. Adding the same amount to both sides preserves the inequality, and an integer write makes it an equality. Any other valid barrier sets $unknown_k$, which stays set until a valid integer write is absorbed and restores equality. While a barrier is in P_k, the claim does not apply. □

Theorem 1 (Escrow safety)**.** *Under A1 to A4 and A6, let a debit d on key k take the fast path. Then (i) its guard holds on the value of k before d in $S(L)$, and (ii) its guard holds on every value of k at which the VC may evaluate d or apply its increment, whatever other accesses of the commutative class to k have been applied by then. Hence every classification yields the outcome of $S(L)$, even if committers classify differently.*

Proof. The proof is part of the induction of Theorem 2: it assumes only that every transaction before d in L is decided as in $S(L)$. By A4, every value x of k that d meets satisfies $0 \leq x \leq 2^{63} - 1$ and $x + \delta_d \leq 2^{63} - 1$, so the guard of d holds on x iff $x + \delta_d \geq \ell_d$. When d was classified, $unknown_k$ was not set, no barrier was in P_k, and $LB(d) + \delta_d \geq \ell_k^* \geq \ell_d$.

(i) Let X be the value of k before d in $S(L)$, and partition the accesses to k before d as they stood when d was classified. The absorbed prefix contributes to $LB(d)$ at most what it contributes to X (Lemma 4). A final access in P_k contributes δ_i to both if it is valid and nothing to either otherwise. A credit that is not final contributes 0 to $LB(d)$ and $\delta_i \geq 0$ or 0 to X. A debit that is not final contributes $\min(\delta_i, 0)$ to $LB(d)$ and δ_i or 0 to X. No other access is in P_k, since it held no barrier. Hence $X \geq LB(d)$ and $X + \delta_d \geq \ell_d$.

(ii) Let V be a value of k at which the VC evaluates d or applies its increment, and let C be the set of valid accesses to k whose effects V contains. Accesses to k before d outside the commutative class conflict with d, so by A1 they are final, and in C if valid, once d is dispatched. So is every access that was absorbed or final when d, or a later debit, was classified. Accesses after d in C do not conflict with d, which is not final, so they are credits or fast-path debits.

Case a: no fast-path debit after d is in C. Compare V with $LB(d)$ as in (i), with V in place of X. The accesses before d contribute as there, a debit that was not final contributes δ_i if it is in C and 0 otherwise, both at least $\min(\delta_i, 0)$, and credits after d contribute at least 0. Hence $V \geq LB(d)$ and $V + \delta_d \geq \ell_d$.

Case b: let d_m be the last fast-path debit after d in C. When d_m was classified, d had been classified and was not final, so $LB(d_m)$ counts d as $\min(\delta_d, 0) \leq \delta_d$, and $\ell_k^* \geq \ell_d$ at that time. Compare $V + \delta_d$ with $LB(d_m) + \delta_{d_m}$ access by access, as in Case a but for the classification of d_m. Accesses before d_m other than d contribute at least as much to V as to $LB(d_m)$, d contributes δ_d to the left side and at most δ_d to the right side, d_m contributes δ_{d_m} to both, and the accesses after d_m in C are credits, which contribute at least 0. Hence $V + \delta_d \geq LB(d_m) + \delta_{d_m} \geq \ell_k^* \geq \ell_d$, with ℓ_k^* taken when d_m was classified.

The two cases cover the value fetched at the start of a batch (Algorithm 2, line 1), which may precede increments of earlier transactions in the same or in concurrent batches, and the value to which the increment of the batch is applied. An ordered-path debit is evaluated with every conflicting access final (A1), hence on its value in $S(L)$, as in the proof of Theorem 2. Both paths therefore give every debit its outcome in $S(L)$, whichever path each committer chooses. □

a) Soft state: A restarted coordinator loses $prefix_k$, P_k, $unknown_k$ and ℓ_k^*. We assume that it resumes once every transaction it dispatched before the crash is final, from a ledger position p before which every transaction is final, and that it processes every transaction from p on in ledger order as above, recording the outcome of those already final and classifying and dispatching the others. It starts every key at $prefix_k = D$, which by A6 is at most the value of k after the accesses before p, and Lemma 4 and Theorem 1 hold for the run from p on with the same proofs. A restart thus sends more debits to the ordered path but admits none whose guard could fail, and committers with different escrow states agree (Theorem 3).

REFERENCES

[1] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proc. EuroSys*, 2018.

[2] E. Androulaki, M. Brandenburger, M. B. Rosenbaum, A. De Caro, K. Elkhiyaoui, A. Filios, L. Funaro, Y. Manevich, H. Meir, S. Natarajan, M. Sethi, and Y. Tock, "Fabric-X: Scaling Hyperledger Fabric for asset exchange," in *Proc. SIGMOD Companion*, 2026, pp. 162–174.

[3] Hyperledger Fabric, "Fabric Gateway: transaction submit flow and MVCC_READ_CONFLICT," https://github.com/hyperledger/fabric-samples/blob/main/full-stack-asset-transfer-guide/docs/ApplicationDev/01-FabricGateway.md, accessed Oct. 2026.

[4] A. Sharma, F. M. Schuhknecht, D. Agrawal, and J. Dittrich, "Blurring the lines between blockchains and database systems: The case of Hyperledger Fabric," in *Proc. SIGMOD*, 2019.

[5] P. Ruan, D. Loghin, Q.-T. Ta, M. Zhang, G. Chen, and B. C. Ooi, "A transactional perspective on execute-order-validate blockchains," in *Proc. SIGMOD*, 2020.

[6] Hyperledger Fabric-X, "fabric-x-evm: EVM execution for Hyperledger Fabric-X," https://github.com/hyperledger/fabric-x-evm, 2026.

[7] C. Gorenflo, L. Golab, and S. Keshav, "XOX Fabric: A hybrid approach to blockchain transaction execution," in *Proc. ICBC*, 2020.

[8] J. Song, J. Jeong, J. Lee, I. Na, and M.-S. Kim, "HTFabric: A fast re-ordering and parallel re-execution method for a high-throughput blockchain," in *Proc. CIKM*, 2024.

[9] M. Cao, B. Cao, W. Peng, and M. Peng, "Revisiting OCC in permissioned blockchain via fast re-execution," in *Proc. INFOCOM*, 2026.

[10] ——, "Overpass Ledger: Full parallelization and fast re-execution for high-performance permissioned blockchain," *IEEE Trans. Parallel Distrib. Syst.*, vol. 37, no. 8, 2026.

[11] P. Nasirifard, R. Mayer, and H.-A. Jacobsen, "FabricCRDT: A conflict-free replicated datatypes approach to permissioned blockchains," in *Proc. Middleware*, 2019.

[12] Hyperledger Fabric, "High-throughput network sample," https://github.com/hyperledger/fabric-samples/tree/main/high-throughput, accessed Nov. 2026.

[13] Oracle, "Oracle Blockchain Platform: MVCC optimization," Oracle Blockchain Platform documentation and Blockchain App Builder 21.3.2, accessed Oct. 2026.

[14] P. E. O'Neil, "The escrow transactional method," *ACM Trans. Database Syst.*, vol. 11, no. 4, pp. 405–430, 1986.

[15] V. Balegas, D. Serra, S. Duarte, C. Ferreira, M. Shapiro, R. Rodrigues, and N. Preguiça, "Extending eventually consistent cloud databases for enforcing numeric invariants," in *Proc. SRDS*, 2015.

[16] D. Gawlick and D. Kinkade, "Varieties of concurrency control in IMS/VS Fast Path," *IEEE Database Eng. Bull.*, vol. 8, no. 2, pp. 3–10, 1985.

[17] Aptos Foundation, "AIP-47: Aggregators V2," Aptos Improvement Proposal, https://github.com/aptos-foundation/AIPs/blob/main/aips/aip-047-aggregators-v2.md, 2023.

[18] M. Dashti, S. B. John, A. Shaikhha, and C. Koch, "Transaction repair for multi-version concurrency control," in *Proc. SIGMOD*, 2017.

[19] Y. Wu, C.-Y. Chan, and K.-L. Tan, "Transaction healing: Scaling optimistic concurrency control on multicores," in *Proc. SIGMOD*, 2016.

[20] H. Lin, H. Feng, Y. Zhou, and L. Wu, "ParallelEVM: Operation-level concurrent transaction execution for EVM-compatible blockchains," in *Proc. EuroSys*, 2025.

[21] Y. Chen, Z. Guo, R. Li, S. Chen, L. Zhou, Y. Zhou, and X. Zhang, "Forerunner: Constraint-based speculative transaction execution for Ethereum," in *Proc. SOSP*, 2021.

[22] A. Thomson, T. Diamond, S.-C. Weng, K. Ren, P. Shao, and D. J. Abadi, "Calvin: Fast distributed transactions for partitioned database systems," in *Proc. SIGMOD*, 2012.

[23] Y. Lu, X. Yu, L. Cao, and S. Madden, "Aria: A fast and practical deterministic OLTP database," *Proc. VLDB Endow.*, vol. 13, no. 11, pp. 2047–2060, 2020.

[24] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, and R. Zhou, "Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing," in *Proc. PPoPP*, 2023.

[25] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, "BLOCKBENCH: A framework for analyzing private blockchains," in *Proc. SIGMOD*, 2017.

[26] T. T. A. Dinh, R. Liu, M. Zhang, G. Chen, B. C. Ooi, and J. Wang, "Untangling blockchain: A data processing view of blockchain systems," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 7, pp. 1366–1385, 2018.

[27] Z. Ge, D. Loghin, B. C. Ooi, P. Ruan, and T. Wang, "Hybrid blockchain database systems: Design and performance," *Proc. VLDB Endow.*, vol. 15, no. 5, pp. 1092–1104, 2022.

[28] Z. Ma, X. Li, H. Zhao, T. Zhou, N. Sheng, and H. Cheng, "BanknoteChain: Achieving user-incentivized parallelism in blockchain via a banknote-inspired transaction model," in *Proc. ICDE*, 2026.

[29] J. A. Chacko, R. Mayer, and H.-A. Jacobsen, "Why do my blockchain transactions fail? A study of Hyperledger Fabric," in *Proc. SIGMOD*, 2021.

[30] B.-J. Ji and T.-W. Kuo, "Better clients, less conflicts: Hyperledger Fabric conflict avoidance," in *Proc. ICBC*, 2024.

[31] S. Zhao, Z. Zhang, J. Wang, Y. Yuan, M. Zhang, G. Wang, and J. Xiao, "Efficient partial order based transaction processing for permissioned blockchains," in *Proc. ICDE*, 2024.

[32] C. Gorenflo, S. Lee, L. Golab, and S. Keshav, "FastFabric: Scaling Hyperledger Fabric to 20,000 transactions per second," in *Proc. ICBC*, 2019.

[33] P. Thakkar and S. Natarajan, "Scaling blockchains using pipelined execution and sparse peers," in *Proc. SoCC*, 2021.

[34] C. Wu, M. J. Amiri, J. Asch, H. Nagda, Q. Zhang, and B. T. Loo, "FlexChain: An elastic disaggregated blockchain," *Proc. VLDB Endow.*, vol. 16, no. 1, pp. 23–36, 2022.

[35] C. Wu, B. Mehta, M. J. Amiri, R. Marcus, and B. T. Loo, "AdaChain: A learned adaptive blockchain," *Proc. VLDB Endow.*, vol. 16, no. 8, pp. 2033–2046, 2023.

[36] R. Huang, Y. Ye, G. Wang, M. Wan, Y. Duan, J. Zhang, G. Xue, and S. Liu, "Back to the future: Rethinking endorsement in order-execute blockchains," arXiv:2604.27659, 2026.

[37] A. Reuter, "Concurrency on high-traffic data elements," in *Proc. PODS*, 1982, pp. 83–92.

[38] Z. Guo, K. Wu, C. Yan, and X. Yu, "Releasing locks as early as you can: Reducing contention of hotspots by violating two-phase locking," in *Proc. SIGMOD*, 2021.

[39] C. Ye, W.-C. Hwang, K. Chen, and X. Yu, "Polaris: Enabling transaction priority in optimistic concurrency control," in *Proc. SIGMOD*, 2023.

[40] W. E. Weihl, "Commutativity-based concurrency control for abstract data types," *IEEE Trans. Comput.*, vol. 37, no. 12, pp. 1488–1505, 1988.

[41] S. Roy, L. Kot, G. Bender, B. Ding, H. Hojjat, C. Koch, N. Foster, and J. Gehrke, "The homeostasis protocol: Avoiding transaction coordination through program analysis," in *Proc. SIGMOD*, 2015.

[42] P. Bailis, A. Fekete, M. J. Franklin, A. Ghodsi, J. M. Hellerstein, and I. Stoica, "Coordination avoidance in database systems," *Proc. VLDB Endow.*, vol. 8, no. 3, pp. 185–196, 2014.

[43] C. Li, D. Porto, A. Clement, J. Gehrke, N. Preguiça, and R. Rodrigues, "Making geo-replicated systems fast as possible, consistent when necessary," in *Proc. OSDI*, 2012.

[44] C. Li, J. Leitão, A. Clement, N. Preguiça, R. Rodrigues, and V. Vafeiadis, "Automating the choice of consistency levels in replicated systems," in *Proc. USENIX ATC*, 2014.

[45] C. Blundell, A. Raghavan, and M. M. K. Martin, "RETCON: Transactional repair without replay," in *Proc. ISCA*, 2010, pp. 258–269.

[46] P. A. Bernstein, C. W. Reid, M. Wu, and X. Yuan, "Optimistic concurrency control by melding trees," *Proc. VLDB Endow.*, vol. 4, no. 11, pp. 944–955, 2011.

[47] N. Narula, C. Cutler, E. Kohler, and R. Morris, "Phase reconciliation for contended in-memory transactions," in *Proc. OSDI*, 2014.

[48] J. He, Y. Liu, J. Chen, S. Peng, Y. Wang, and M. Zhang, "MoryFabric: Reducing transaction abort by actual validity verification and reordering," in *Proc. ICPADS*, 2023.

[49] J. Wen, W. Chen, J. Zhang, X. Chen, T. Cai, H.-N. Dai, and Z. Zheng, "RapidSnail: Improve scalability of blockchain under high contention workload," *IEEE Trans. Comput.*, vol. 75, no. 1, pp. 261–274, 2026.

[50] M. J. Amiri, D. Agrawal, and A. El Abbadi, "ParBlockchain: Leveraging transaction parallelism in permissioned blockchain systems," in *Proc. ICDCS*, 2019.

[51] X. Qi, Z. Chen, H. Zhuo, Q. Xu, C. Zhu, Z. Zhang, C. Jin, A. Zhou, Y. Yan, and H. Zhang, "SChain: Scalable concurrency over flexible permissioned blockchain," in *Proc. ICDE*, 2023.

[52] Z. Peng, Y. Zhang, Q. Xu, H. Liu, Y. Gao, X. Li, and G. Yu, "NeuChain: A fast permissioned blockchain system with deterministic ordering," *Proc. VLDB Endow.*, vol. 15, no. 11, pp. 2585–2598, 2022.

[53] Z. Chen, T. Yang, Y. Zheng, Z. Zhang, C. Jin, and A. Zhou, "Spectrum: Speedy and strictly-deterministic smart contract transactions for blockchain ledgers," *Proc. VLDB Endow.*, vol. 17, no. 10, pp. 2541–2554, 2024.

[54] Z. Liu, A. Sonnino, I. Zablotchi, E. Kokoris-Kogias, and M. Kogias, "Remora: Scale-out deterministic execution for smart contracts," *Proc. VLDB Endow.*, vol. 19, pp. 3076–3090, 2026.

[55] X. Tong, Z. Ye, Z. Zhang, C. Jin, and A. Zhou, "TELL: Efficient transaction execution protocol towards leaderless consensus," in *Proc. ICDE*, 2024.

[56] W. Chen, D. Xia, Z. Cai, H.-N. Dai, J. Zhang, Z. Hong, J. Liang, and Z. Zheng, "Porygon: Scaling blockchain via 3D parallelism," in *Proc. ICDE*, 2024.

[57] H. Zhang, X. Qi, H. Tang, Z. Zhang, C. Jin, and A. Zhou, "Loom: A deterministic execution framework towards nested contract transactions," in *Proc. ICDE*, 2025.

[58] S. Zhang, R. Cheng, X. Liu, J. Xiao, H. Jin, and B. Li, "Seer: Accelerating blockchain transaction execution by fine-grained branch prediction," *Proc. VLDB Endow.*, vol. 18, no. 3, pp. 822–835, 2025.

[59] J. Wei, X. Qi, Q. Que, Z. Zhang, Y. Yang, and C. Jin, "Chubby: Robust smart contract execution against dependency over-declaration," in *Proc. ICDE*, 2026.

[60] S. Nathan, C. Govindarajan, A. Saraf, M. Sethi, and P. Jayachandran, "Blockchain meets database: Design and implementation of a blockchain relational database," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1539–1552, 2019.

[61] J. Newsome and D. Song, "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software," in *Proc. NDSS*, 2005.

[62] P. Godefroid, N. Klarlund, and K. Sen, "DART: Directed automated random testing," in *Proc. PLDI*, 2005, pp. 213–223.

[63] W. Tang, "EIP-2200: Structured definitions for net gas metering," Ethereum Improvement Proposals, https://eips.ethereum.org/EIPS/eip-2200, 2019.

[64] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Yellow Paper, 2014.

[65] M. Alomari, M. J. Cahill, A. Fekete, and U. Röhm, "The cost of serializability on platforms that use snapshot isolation," in *Proc. ICDE*, 2008, pp. 576–585.

[66] H. Adams, N. Zinsmeister, and D. Robinson, "Uniswap v2 core," Whitepaper, https://app.uniswap.org/whitepaper.pdf, 2020.

[67] X. Yu, G. Bezerra, A. Pavlo, S. Devadas, and M. Stonebraker, "Staring into the abyss: An evaluation of concurrency control with one thousand cores," *Proc. VLDB Endow.*, vol. 8, no. 3, pp. 209–220, 2014.

[68] S. Gupta, S. Rahnama, J. Hellings, and M. Sadoghi, "ResilientDB: Global scale resilient blockchain fabric," *Proc. VLDB Endow.*, vol. 13, no. 6, pp. 868–883, 2020.